

Practical-session-6

Task-1

Create a client-server application using Named Pipes (FIFOs). The client sends messages and the Server processes and responds to them. Analyze FIFO behavior when multiple clients communicate simultaneously.

Nanoserver.c

```
root@Shamikahasin: ~/fifo_f x + v
GNU nano 7.2 server.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>

#define REQUEST_FIFO "request_fifo"
#define RESPONSE_FIFO "response_fifo"

int main()
{
    char message[100];
    char response[200];

    // Create named pipes
    mkfifo(REQUEST_FIFO, 0666);
    mkfifo(RESPONSE_FIFO, 0666);

    printf("Server started. Waiting for clients...\n");

    while (1)
    {
        // Open request FIFO for reading
        int request_fd = open(REQUEST_FIFO, O_RDONLY);

        if (request_fd < 0)
        {
            perror("open request_fifo");
            exit(1);
        }

        // Read client message
        int n = read(request_fd, message, sizeof(message) - 1);

        if (n > 0)
        {
            message[n] = '\0';

            printf("Client sent: %s\n", message);

            message[n] = '\0';

            printf("Client sent: %s\n", message);

            // Process message
            if (strcmp(message, "hello") == 0)
            {
                strcpy(response, "Hello Client!");
            }
            else if (strcmp(message, "time") == 0)
            {
                strcpy(response, "Server received your time request.");
            }
            else if (strcmp(message, "exit") == 0)
            {
                strcpy(response, "Goodbye!");
            }
            else
            {
                snprintf(response, sizeof(response),
                    "Server processed: %s", message);
            }

            // Open response FIFO
            int response_fd = open(RESPONSE_FIFO, O_WRONLY);

            write(response_fd, response, strlen(response) + 1);

            close(response_fd);
        }

        close(request_fd);
    }

    return 0;
}
```

nano_client.c

```

GNU nano 7.2 clien
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>

#define REQUEST_FIFO "request_fifo"
#define RESPONSE_FIFO "response_fifo"

int main()
{
    char message[100];
    char response[200];

    printf("Enter message: ");
    fgets(message, sizeof(message), stdin);

    // Remove newline
    message[strcspn(message, "\n")] = '\0';

    // Open request FIFO
    int request_fd = open(REQUEST_FIFO, O_WRONLY);

    if (request_fd < 0)
    {
        perror("open request_fifo");
        exit(1);
    }

    // Send message to server
    write(request_fd, message, strlen(message) + 1);

    close(request_fd);

    // Open response FIFO
    int response_fd = open(RESPONSE_FIFO, O_RDONLY);

    if (response_fd < 0)
    {
        perror("open response_fifo");
        exit(1);
    }

    // Receive server response
    int n = read(response_fd, response, sizeof(response) - 1);

    if (n > 0)
    {
        response[n] = '\0';
        printf("Server response: %s\n", response);
    }

    close(response_fd);

    return 0;
}

```

Execution of client and server

```
root@Shamikahasini: ~/fifo_f x + v
Welcome to Ubuntu 24.04.1 LTS (GNU/Linux 6.18.33.2-microsoft-standard-WSL2 x
86_64)

* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:       https://ubuntu.com/pro

System information as of Thu Sep 10 04:38:49 UTC 2026

System load:  0.78          Processes:           66
Usage of /:   0.2% of 1006.85GB Users logged in:       0
Memory usage: 5%          IPv4 address for eth0: 172.23.238.51
Swap usage:   0%

* Canonical Workshop gives developers fast, composable, reproducible, and
secure developer environments that are perfect for agentic workflows.

https://ubuntu.com/workshop

This message is shown once a day. To disable it please create the
/root/.hushlogin file.
root@Shamikahasini:~# mkdir fifo_project
cd fifo_project
root@Shamikahasini:~/fifo_project# nano server.c
root@Shamikahasini:~/fifo_project# nano client.c
root@Shamikahasini:~/fifo_project# gcc server.c -o server
gcc client.c -o client
root@Shamikahasini:~/fifo_project# ls
client client.c server server.c
root@Shamikahasini:~/fifo_project# ./server
Server started. Waiting for clients...
hello
Client sent: hello

root@Shamikahasini:~# cd ~/fifo_project
root@Shamikahasini:~/fifo_project# ./client
Enter message: hello
Server response: Hello Client!
root@Shamikahasini:~/fifo_project#
```

Observation:

The client and server processes successfully communicated using **Named Pipes (FIFOs)**. The client sent messages through the request FIFO, and the server read, processed, and sent responses through the response FIFO.

When multiple clients communicated simultaneously, their messages were received through the shared FIFO according to the order in which they were processed by the operating system.

TASK-2

Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSR1. Demonstrate asynchronous event handling and explain the role of signal handlers.

```
root@Shamikahasini: ~/fifo_f x + v
GNU nano 7.2 sig
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>

void signal_handler(int sig)
{
    if (sig == SIGINT)
        printf("\nSIGINT received! (Ctrl+C pressed)\n");

    else if (sig == SIGTERM)
        printf("\nSIGTERM received! Termination requested.\n");

    else if (sig == SIGUSR1)
        printf("\nSIGUSR1 received! User-defined event occurred.\n");
}

int main()
{
    // Register signal handlers
    signal(SIGINT, signal_handler);
    signal(SIGTERM, signal_handler);
    signal(SIGUSR1, signal_handler);

    printf("Process ID (PID): %d\n", getpid());
    printf("Waiting for signals...\n");

    while (1)
    {
        printf("Program is running...\n");
        sleep(3);
    }

    return 0;
}
```

```
root@Shamikahasini: ~/signal_project x + v
root@Shamikahasini:~/signal_project# nano signal.c
root@Shamikahasini:~/signal_project# gcc signal.c -o signal
root@Shamikahasini:~/signal_project# ls
signal signal.c
root@Shamikahasini:~/signal_project# ./signal
Process ID (PID): 1410
Waiting for signals...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
SIGINT received! (Ctrl+C pressed)
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
SIGTERM received! Termination requested.
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
SIGUSR1 received! User-defined event occurred.
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Program is running...
Killed
root@Shamikahasini:~/signal_project# |

root@Shamikahasini: ~/signal_project x + v
root@Shamikahasini:~/signal_project# cd ~/signal_project
root@Shamikahasini:~/signal_project# kill -SIGINT 1410
root@Shamikahasini:~/signal_project# kill -SIGTERM 1410
root@Shamikahasini:~/signal_project# kill -SIGUSR1 1410
root@Shamikahasini:~/signal_project# kill -9 1410
root@Shamikahasini:~/signal_project#
```

Observation:

The program successfully captured and handled **SIGINT, SIGTERM, and SIGUSR1** signals using POSIX signal handlers. Signals were delivered asynchronously while the program was running, and the registered signal handler executed immediately when a signal was received.

This demonstrated how signal handlers allow a process to respond to asynchronous events without continuously checking for them.